

Laboratoire d'automatisme et d'informatique industrielle

Mise en place d'une architecture de communication industrielle sous Docker

Node-RED – OPC UA – MQTT – InfluxDB 3 – TimescaleDB – Grafana

Environnement : Windows 11 / WSL2 / Ubuntu 24.04
Conteneurisation : Docker Desktop
Plateforme : Lenovo LOQ
Date : Septembre–Octobre 2026

Documentation technique et pédagogique

Table des matières

1	Introduction	3
2	Architecture générale	3
2.1	Principe de fonctionnement	3
2.2	Répartition des rôles	4
3	Environnement Docker	4
3.1	Principe de la conteneurisation	4
3.2	Création du réseau	4
4	Installation et configuration de Node-RED	4
4.1	Présentation	4
4.2	Dossier persistant	4
4.3	Démarrage du conteneur	5
4.4	Nœuds supplémentaires	5
5	Installation du broker MQTT Mosquitto	5
5.1	Présentation du protocole MQTT	5
5.2	Préparation des dossiers	5
5.3	Configuration utilisée	5
5.4	Démarrage du conteneur	6
5.5	Vérification de fonctionnement	6
6	Mise en place du simulateur OPC UA	6
6.1	Rappel sur OPC UA	6
6.2	Simulateur utilisé	6
6.3	Commande de démarrage	6
7	Connexion Node-RED au serveur OPC UA	7
7.1	Paramètres de sécurité	7
7.2	Navigation dans l'espace d'adressage	7
8	Lecture d'une variable industrielle	8
8.1	Variable sélectionnée	8
8.2	Configuration du flow	8
8.3	Acquisition périodique	8
9	Transmission OPC UA vers MQTT	9
9.1	Principe	9
9.2	Configuration MQTT	9
9.3	Architecture finale du flow	9
9.4	Vérification de la réception MQTT	10
10	Historisation avec InfluxDB 3 et TimescaleDB	10
10.1	InfluxDB 3	10
10.2	TimescaleDB	11
10.3	Validation croisée des deux bases	11

11 Supervision industrielle avec Grafana	12
11.1 Connexion aux sources de données	12
11.2 Comparaison InfluxDB 3 / TimescaleDB	12
11.3 Dashboard de supervision V1	13
11.4 Âge de la dernière mesure	13
11.5 État d’acquisition	14
11.6 État global du laboratoire	14
11.7 Fréquence et comptage des acquisitions	14
12 Orchestration avec Docker Compose	15
12.1 Objectif	15
12.2 Validation de la configuration	15
12.3 Utilisation quotidienne	16
13 Sauvegarde du laboratoire	16
13.1 Dossier de sauvegarde	16
13.2 Archive et copie Windows	17
14 Problèmes rencontrés et solutions	17
14.1 Connexion MQTT impossible	17
14.2 Connexion OPC UA	17
14.3 Erreur lors du parcours OPC UA	18
14.4 Absence de valeurs dans MQTT	18
14.5 Décalage temporel InfluxDB 3	18
14.6 Mélange des variables dans un topic MQTT	18
15 Sécurisation du laboratoire	18
15.1 Limitation de l’exposition réseau	18
15.2 Contrôle des ports	19
15.3 Limites de sécurité	19
16 Commandes utiles de maintenance	19
16.1 Visualiser les conteneurs	19
16.2 Consulter les journaux Node-RED	19
16.3 Consulter les journaux OPC PLC	19
16.4 Contrôler le réseau industriel	20
16.5 Arrêter les services	20
16.6 Redémarrer les services existants	20
17 Bilan des validations	20
18 Perspectives	20
19 Conclusion	21

1 Introduction

L'objectif de ce laboratoire est de mettre en place un environnement local permettant de simuler un système d'acquisition de données industrielles.

L'installation repose désormais sur six services principaux : un simulateur OPC UA, Node-RED, le broker MQTT Mosquitto, InfluxDB 3, TimescaleDB et Grafana.

Cette architecture permet d'étudier plusieurs notions rencontrées en automatisme et en informatique industrielle : communication client-serveur, identification des variables OPC UA, acquisition périodique, publication MQTT, historisation de séries temporelles, supervision et orchestration de services conteneurisés.

Le laboratoire a été étendu progressivement jusqu'à obtenir une chaîne complète d'acquisition, de stockage et de visualisation, pilotée avec Docker Compose et sauvegardée de manière reproductible.

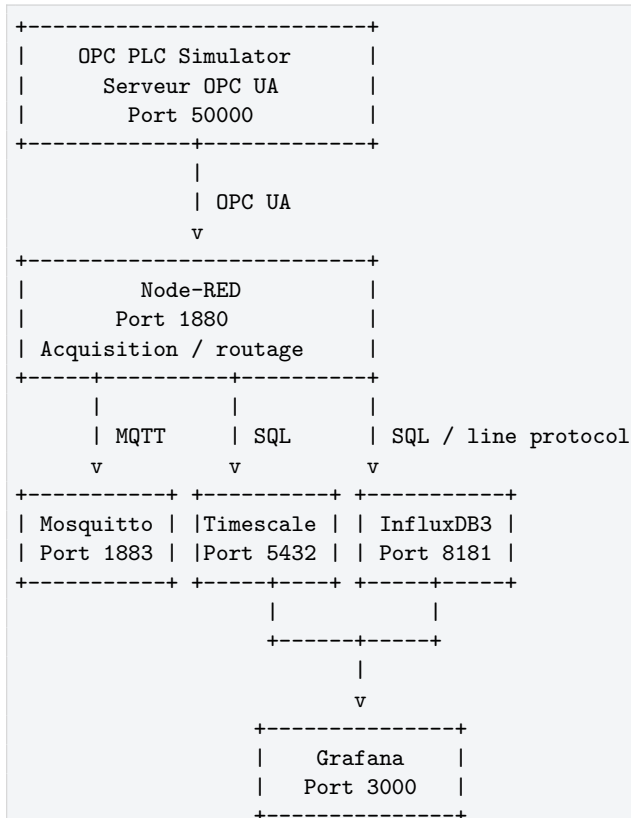
2 Architecture générale

2.1 Principe de fonctionnement

Le simulateur OPC PLC reproduit le comportement d'un serveur OPC UA industriel.

Node-RED joue le rôle de client OPC UA. Il se connecte au simulateur, explore son espace d'adressage et lit les variables sélectionnées.

Les valeurs acquises sont ensuite publiées dans Mosquitto à l'aide du protocole MQTT.



2.2 Répartition des rôles

Composant	Fonction
OPC PLC	Génération de variables simulées accessibles via OPC UA.
Node-RED	Lecture OPC UA, formatage, routage MQTT et écriture vers les bases.
Mosquitto	Réception et distribution des messages MQTT.
InfluxDB 3	Historisation orientée séries temporelles.
TimescaleDB	Historisation temporelle sur PostgreSQL.
Grafana	Supervision, indicateurs et courbes historiques.
Docker Compose	Orchestration des six services du laboratoire.

TABLE 1 – Rôle des composants industriels

3 Environnement Docker

3.1 Principe de la conteneurisation

Docker permet d'exécuter plusieurs applications indépendamment, sans avoir à les installer directement dans le système d'exploitation.

Chaque application dispose de son propre conteneur. La communication entre les conteneurs s'effectue grâce à un réseau Docker commun.

Dans notre laboratoire, ce réseau porte le nom `industrial-net`.

3.2 Création du réseau

```
docker network create industrial-net
```

Pour vérifier les conteneurs connectés :

```
docker network inspect industrial-net \
--format '{{range .Containers}}{{.Name}} {{end}}'
```

Les services industriels sont reliés au même réseau `industrial-net`. Ils peuvent ainsi utiliser leurs noms de conteneurs comme noms d'hôtes, par exemple `opcplc`, `mosquitto`, `influxdb3` et `timescaledb`.

4 Installation et configuration de Node-RED

4.1 Présentation

Node-RED est un environnement de programmation graphique fondé sur des flux.

Il permet de connecter différents protocoles, de traiter des messages et de développer des applications d'intégration industrielle.

Son interface est accessible depuis un navigateur Web.

4.2 Dossier persistant

Un dossier est utilisé afin de conserver les flows, les paramètres et les nœuds supplémentaires.

```
mkdir -p ~/docker_data/node-red
```

4.3 Démarrage du conteneur

La configuration finale du laboratoire est la suivante :

```
docker run -d \  
  --name node-red \  
  --network industrial-net \  
  -p 127.0.0.1:1880:1880 \  
  -v ~/docker_data/node-red:/data \  
  nodered/node-red:latest
```

L'interface est accessible à l'adresse :

<http://localhost:1880>

Le montage du volume garantit que les flows restent disponibles même après la suppression et la recréation du conteneur.

4.4 Nœuds supplémentaires

Le module suivant a été installé depuis le gestionnaire de palette de Node-RED :

```
node-red-contrib-opcua
```

Version constatée lors de l'installation : 0.2.355.

Ce module fournit notamment les nœuds `OpcUa-Browser`, `OpcUa-Item` et `OpcUa-Client`.

5 Installation du broker MQTT Mosquitto

5.1 Présentation du protocole MQTT

MQTT est un protocole de communication reposant sur le modèle publication/abonnement.

Un équipement publie un message sur un sujet (*topic*). Le broker reçoit ce message et le transmet aux clients abonnés au même sujet.

Les applications communicantes n'ont donc pas besoin de se connaître directement.

5.2 Préparation des dossiers

```
mkdir -p ~/docker_data/mosquitto/config  
mkdir -p ~/docker_data/mosquitto/data  
mkdir -p ~/docker_data/mosquitto/log
```

5.3 Configuration utilisée

Le fichier suivant a été créé :

```
~/docker_data/mosquitto/config/mosquitto.conf
```

Son contenu est :

```
listener 1883  
allow_anonymous true  
  
persistence true  
persistence_location /mosquitto/data/  
  
log_dest file /mosquitto/log/mosquitto.log
```

L'option `allow_anonymous true` autorise les connexions sans authentification.

Cette configuration est réservée à notre laboratoire local. Elle ne doit pas être reprise telle quelle dans une installation industrielle exposée.

5.4 Démarrage du conteneur

```
docker run -d \  
  --name mosquitto \  
  --network industrial-net \  
  -p 127.0.0.1:1883:1883 \  
  -v ~/docker_data/mosquitto/config:/mosquitto/config \  
  -v ~/docker_data/mosquitto/data:/mosquitto/data \  
  -v ~/docker_data/mosquitto/log:/mosquitto/log \  
  eclipse-mosquitto:latest
```

Le broker écoute sur le port TCP 1883.

5.5 Vérification de fonctionnement

Deux outils sont disponibles dans le conteneur :

`mosquitto_pub` pour publier un message et `mosquitto_sub` pour recevoir des messages.

La communication MQTT a d'abord été testée avec un message simple :

```
docker exec -it mosquitto \  
  mosquitto_sub -h localhost -t test/topic
```

Dans un deuxième terminal :

```
docker exec -it mosquitto \  
  mosquitto_pub -h localhost \  
  -t test/topic \  
  -m "Bonjour MQTT"
```

Le message a bien été reçu par l'abonné.

6 Mise en place du simulateur OPC UA

6.1 Rappel sur OPC UA

OPC UA signifie *Open Platform Communications Unified Architecture*.

Ce standard est utilisé pour échanger des données entre équipements et applications industrielles.

Il permet notamment de structurer les informations sous forme d'objets et de variables.

Chaque nœud possède un identifiant appelé `NodeId`.

Par exemple :

```
ns=3;s=StepUp
```

Dans cette expression :

- `ns=3` indique l'index du namespace ;
- `s=StepUp` désigne un identifiant textuel.

6.2 Simulateur utilisé

Le laboratoire utilise le simulateur OPC PLC proposé par Microsoft.

Il est exécuté dans un conteneur Docker et expose un serveur OPC UA.

6.3 Commande de démarrage

```
docker run -d \  
  --name opcplc \  
  --hostname opcplc \  
  --network industrial-net \  
  -p 127.0.0.1:50000:50000
```

```
mcr.microsoft.com/iotedgedge/opc-plc:latest \
--pn=50000 \
--plchostname=opcplc \
--certdnsnames=opcplc \
--autoaccept \
--sn=5 \
--sr=10 \
--st=uint \
--fn=5 \
--fr=1 \
--ft=uint \
--gn=5
```

L'endpoint configuré dans Node-RED est :

```
opc.tcp://opcplc:50000
```

Le nom `opcplc` est résolu à l'intérieur du réseau Docker `industrial-net`.

L'option `-autoaccept` simplifie les essais de connexion en acceptant automatiquement les certificats clients. Elle est réservée au laboratoire.

7 Connexion Node-RED au serveur OPC UA

7.1 Paramètres de sécurité

Après les premiers essais, la connexion OPC UA a été établie avec les paramètres suivants :

Paramètre	Valeur
Endpoint	<code>opc.tcp://opcplc:50000</code>
Security Mode	<code>SignAndEncrypt</code>
Security Policy	<code>Basic256Sha256</code>
Identité utilisateur	<code>Anonymous</code>

TABLE 2 – Configuration OPC UA retenue

Le nœud `OpcUa-Client` a ensuite affiché le statut `session active`, confirmant l'établissement de la session.

7.2 Navigation dans l'espace d'adressage

Le nœud `OpcUa-Browser` permet d'explorer les objets exposés par le serveur.

Pour effectuer une exploration, un nœud Inject transmet le `NodeId` du dossier parent dans `msg.topic`.

Le Browser renvoie alors les références de ses éléments enfants.

Le parcours ayant permis de retrouver les données simulées est présenté ci-dessous.

```
Objects
NodeId : ns=0;i=85
|
+-- OpcPlc
   NodeId : ns=3;s=OpcPlc
   |
   +-- Telemetry
      NodeId : ns=3;s=Telemetry
      |
      +-- Basic
         NodeId : ns=3;s=Basic
         |
         +-- StepUp
         +-- RandomSignedInt32
```

```
+-- RandomUnsignedInt32
```

Il est important de distinguer les dossiers **Objects** et **Types** :

```
ObjectsFolder : ns=0;i=85
TypesFolder   : ns=0;i=86
```

Une première exploration avait utilisé `ns=0;i=86`, ce qui renvoyait les types standards OPC UA plutôt que les objets applicatifs.

8 Lecture d'une variable industrielle

8.1 Variable sélectionnée

Pour notre premier essai, nous avons retenu la variable simulée **StepUp**.

Propriété	Valeur
Nom	StepUp
NodeId	<code>ns=3;s=StepUp</code>
Type	UInt32
Classe de nœud	Variable (<code>nodeClass = 2</code>)

TABLE 3 – Caractéristiques de la variable StepUp

Le type `UInt32` correspond à un entier non signé codé sur 32 bits.
Il autorise les valeurs comprises entre 0 et 4 294 967 295.

8.2 Configuration du flow

Le flow de lecture est constitué des éléments suivants :

```
Inject
|
v
OpcUa-Item
|
v
OpcUa-Client (READ)
|
+-----> Debug
```

Dans `OpcUa-Item`, les paramètres sont :

```
Item      : ns=3;s=StepUp
DataType  : UInt32
Value     : vide
```

Le nœud `OpcUa-Client` est configuré avec l'action **READ**.

8.3 Acquisition périodique

Le nœud `Inject` a finalement été configuré avec une répétition d'environ deux secondes.
La fréquence théorique d'acquisition est donc :

$$f = \frac{1}{T}$$

Avec :

$$T \approx 2 \text{ s}$$

On obtient :

$$f \approx 0,5 \text{ Hz}$$

Dans Grafana, la fréquence calculée sur une fenêtre d'une minute est typiquement voisine de 0,47 à 0,50 Hz, ce qui est cohérent avec cette période d'acquisition.

9 Transmission OPC UA vers MQTT

9.1 Principe

Une fois la lecture OPC UA fonctionnelle, les valeurs récupérées ont été transmises au broker MQTT Mosquitto.

Un nœud `mqtt out` a été ajouté au flow.

9.2 Configuration MQTT

Les paramètres retenus sont :

Paramètre	Valeur
Broker	Mosquitto Industrial
Serveur	<code>mosquitto</code>
Port	1883
Protocole	MQTT V3.1.1
Topic	dynamique : <code>industrie/opcua/<variable></code>
QoS	0
Retain	false

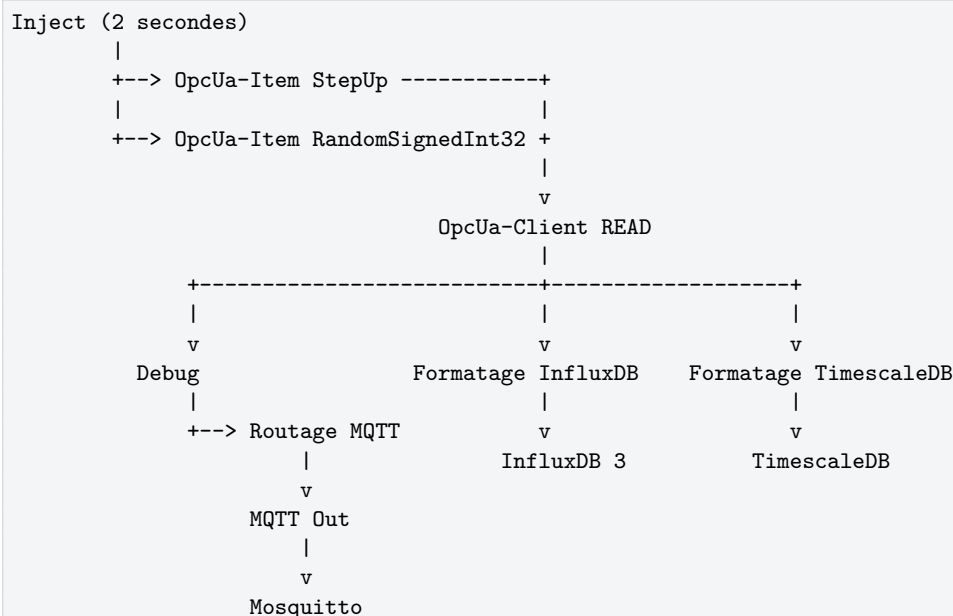
TABLE 4 – Paramètres de publication MQTT

Le QoS 0 permet une publication sans garantie de réception.

L'option Retain désactivée évite de conserver automatiquement la dernière mesure publiée pour les nouveaux abonnés.

9.3 Architecture finale du flow

Deux variables sont lues avec le même client OPC UA : `StepUp` et `RandomSignedInt32`.



Le flow final est illustré à la Figure 1. Il montre le client OPC UA commun aux deux variables, le routage MQTT dynamique ainsi que les deux branches d'historisation vers InfluxDB 3 et TimescaleDB.

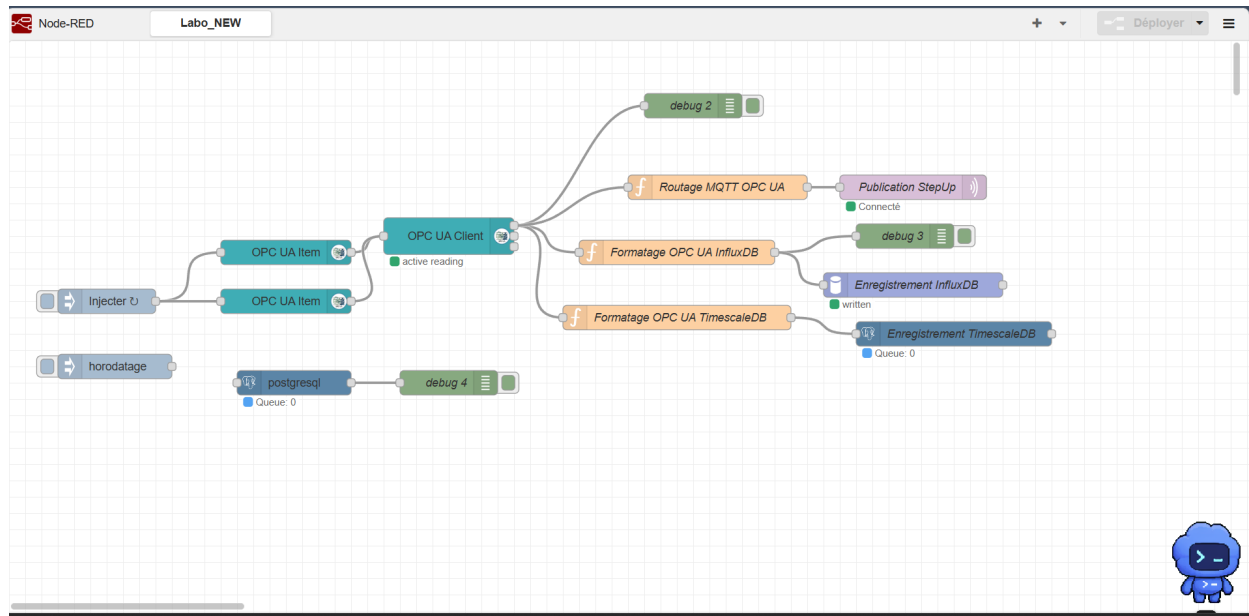


FIGURE 1 – Flow Node-RED final : acquisition OPC UA multi-variable, routage MQTT dynamique et historisation vers InfluxDB 3 et TimescaleDB.

Le routage MQTT est devenu dynamique afin d'éviter de mélanger plusieurs variables dans un même topic. La fonction Node-RED utilisée est :

```
const variable = String(msg.topic || "").split(";s=")[1] || "unknown";
msg.topic = 'industrie/opcu/${variable}';
return msg;
```

9.4 Vérification de la réception MQTT

Afin de confirmer la transmission, un client MQTT en ligne de commande a été utilisé.

```
docker exec -it mosquitto \
  mosquitto_sub -h localhost \
  -t industrie/opcu/StepUp -C 5
```

```
docker exec -it mosquitto \
  mosquitto_sub -h localhost \
  -t industrie/opcu/RandomSignedInt32 -C 5
```

L'option `-C 5` demande au client de recevoir cinq messages avant de quitter automatiquement.

Les essais ont confirmé que les deux variables sont publiées sur des topics séparés. Pour `StepUp`, les valeurs reçues augmentent régulièrement, tandis que `RandomSignedInt32` produit des valeurs signées variables.

10 Historisation avec InfluxDB 3 et TimescaleDB

10.1 InfluxDB 3

InfluxDB 3 Core est exécuté dans un conteneur Docker sur le port local 8181. La base utilisée est `industrial_lab`.

La mesure générique retenue est `opcu`. Les variables sont différenciées par un tag `variable`, ce qui évite de créer une measurement différente pour chaque `NodeId`.

Le formatage Node-RED utilisé est :

```
const valeur = Number(msg.payload);
if (!Number.isFinite(valeur)) return null;

const variable = String(msg.topic || "").split(";s=")[1] || "unknown";
msg.timestamp = new Date(msg.sourceTimestamp);

msg.payload = {
  fields: { value: valeur },
  tags: {
    machine: "opcplc",
    variable: variable
  }
};
return msg;
```

L'usage explicite de `msg.sourceTimestamp` est important : il garantit que l'horodatage enregistré correspond au timestamp fourni par OPC UA.

Une requête de contrôle typique est :

```
SELECT time, variable, value
FROM opcua
ORDER BY time DESC
LIMIT 10;
```

10.2 TimescaleDB

TimescaleDB est utilisé avec PostgreSQL 17. La base s'appelle également `industrial_lab`. La table principale est :

```
CREATE TABLE IF NOT EXISTS mesures_opcua (
  time TIMESTAMPTZ NOT NULL,
  machine TEXT NOT NULL,
  variable TEXT NOT NULL,
  value DOUBLE PRECISION NOT NULL
);
```

Elle a été transformée en hypertable TimescaleDB.

La fonction Node-RED prépare les paramètres SQL à partir du timestamp OPC UA :

```
const valeur = Number(msg.payload);
if (!Number.isFinite(valeur)) return null;

const variable = String(msg.topic || "").split(";s=")[1] || "unknown";
const horodatage = new Date(msg.sourceTimestamp).toISOString();

msg.params = [horodatage, "opcplc", variable, valeur];
return msg;
```

L'insertion est réalisée avec :

```
INSERT INTO mesures_opcua
  (time, machine, variable, value)
VALUES
  ($1, $2, $3, $4);
```

10.3 Validation croisée des deux bases

Les dernières valeurs de `StepUp` et `RandomSignedInt32` ont été retrouvées dans les deux bases avec des timestamps cohérents et une cadence d'environ deux secondes.

Cette redondance permet de comparer deux approches : une base de séries temporelles dédiée avec InfluxDB 3 et une base relationnelle temporelle avec TimescaleDB.

11 Supervision industrielle avec Grafana

11.1 Connexion aux sources de données

Grafana est accessible localement sur le port 3000. Deux sources de données ont été configurées : InfluxDB 3 et TimescaleDB.

Les requêtes historiques filtrent la plage temporelle sélectionnée dans Grafana avec `$__timeFilter(time)`.

Exemple pour TimescaleDB :

```
SELECT
  time,
  value
FROM mesures_opcua
WHERE $__timeFilter(time)
  AND variable = 'StepUp'
ORDER BY time ASC;
```

Exemple pour InfluxDB 3 :

```
SELECT
  time,
  value
FROM opcua
WHERE $__timeFilter(time)
  AND variable = 'StepUp'
ORDER BY time ASC;
```

11.2 Comparaison InfluxDB 3 / TimescaleDB

Le premier dashboard a permis de comparer visuellement les mêmes variables dans les deux bases.

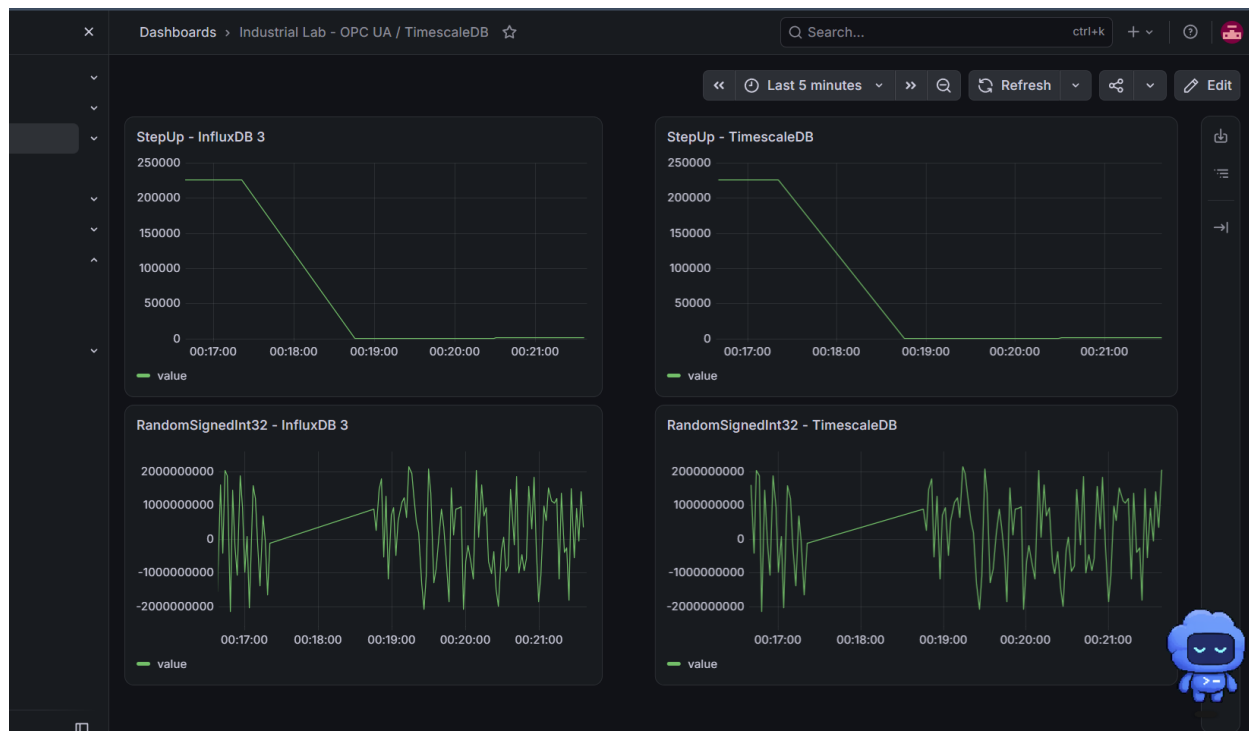


FIGURE 2 – Comparaison des historiques StepUp et RandomSignedInt32 dans InfluxDB 3 et TimescaleDB.

Après un redémarrage du simulateur OPC PLC, la variable `StepUp` repart de zéro. Grafana peut alors relier le dernier point avant arrêt au premier point après redémarrage, ce qui explique

certaines grandes lignes diagonales observées dans les historiques.

11.3 Dashboard de supervision V1

Le dashboard a ensuite été enrichi avec des indicateurs plus proches d'une supervision industrielle :

- état global du laboratoire ;
- âge de la dernière mesure StepUp ;
- état d'acquisition StepUp ;
- fréquence d'acquisition ;
- nombre de mesures reçues sur une minute ;
- valeur instantanée de StepUp ;
- valeur instantanée de RandomSignedInt32 ;
- historiques comparatifs InfluxDB 3 / TimescaleDB.

Le rafraîchissement automatique du dashboard est réglé à 5 s.

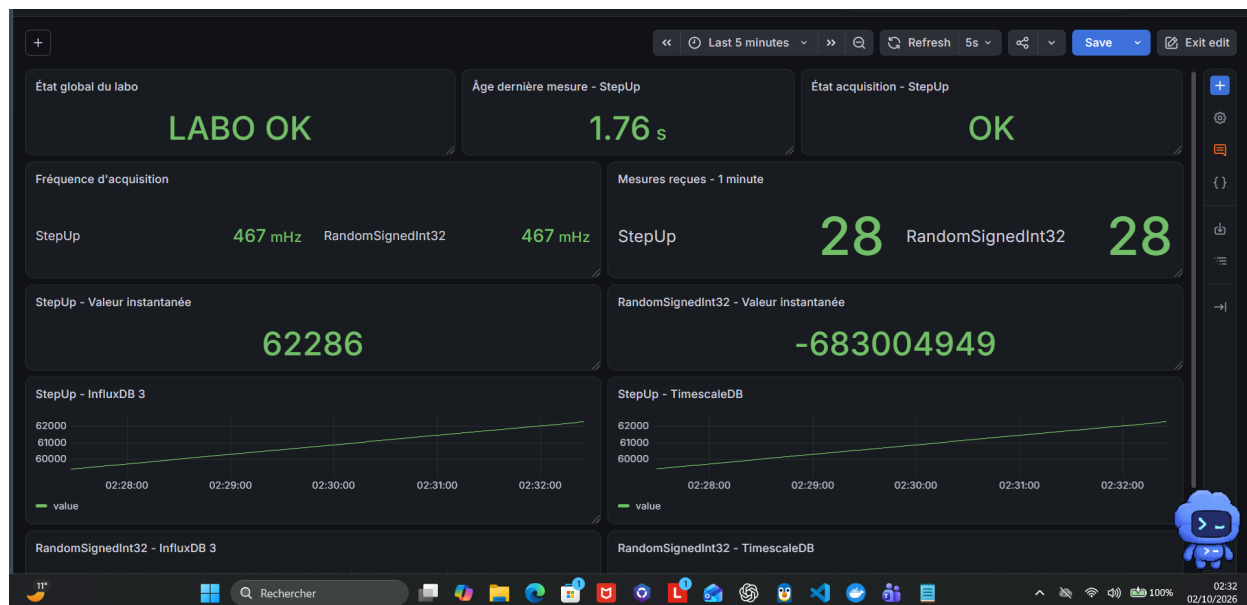


FIGURE 3 – Dashboard Grafana de supervision industrielle V1.

11.4 Âge de la dernière mesure

L'âge de la dernière mesure est calculé avec :

```
SELECT
  EXTRACT(EPOCH FROM (NOW() - MAX(time))) AS age_seconds
FROM mesures_opcua
WHERE variable = 'StepUp';
```

Les seuils visuels retenus sont :

- vert : moins de 5 secondes ;
- orange : à partir de 5 secondes ;
- rouge : à partir de 10 secondes.

11.5 État d'acquisition

L'état StepUp est calculé numériquement afin de faciliter le mapping de valeurs dans Grafana :

```
SELECT
CASE
  WHEN NOW() - MAX(time) < INTERVAL '10 seconds'
  THEN 1
  ELSE 0
END AS acquisition_status
FROM mesures_opcua
WHERE variable = 'StepUp';
```

Le mapping Grafana associe 1 à OK en vert et 0 à PERTE DE DONNEES en rouge.

11.6 État global du laboratoire

Le laboratoire est considéré comme disponible si les deux variables sont récentes :

```
SELECT
CASE
  WHEN
    NOW() - MAX(time) FILTER (WHERE variable = 'StepUp')
    < INTERVAL '10 seconds'
  AND
    NOW() - MAX(time) FILTER (WHERE variable = 'RandomSignedInt32')
    < INTERVAL '10 seconds'
  THEN 1
  ELSE 0
END AS lab_status
FROM mesures_opcua
WHERE variable IN ('StepUp', 'RandomSignedInt32');
```

Le mapping associé affiche LABO OK en vert ou DEFAUT ACQUISITION en rouge.

11.7 Fréquence et comptage des acquisitions

La fréquence moyenne sur la dernière minute est calculée par :

```
SELECT
COUNT(*) FILTER (WHERE variable = 'StepUp') / 60.0
AS "StepUp",
COUNT(*) FILTER (WHERE variable = 'RandomSignedInt32') / 60.0
AS "RandomSignedInt32"
FROM mesures_opcua
WHERE time >= NOW() - INTERVAL '1 minute';
```

Une valeur observée d'environ 0,467 Hz correspond à environ 28 mesures par minute, ce qui est cohérent avec une acquisition proche de deux secondes.

Le nombre de mesures reçues est obtenu avec :

```
SELECT
COUNT(*) FILTER (WHERE variable = 'StepUp')
AS "StepUp",
COUNT(*) FILTER (WHERE variable = 'RandomSignedInt32')
AS "RandomSignedInt32"
FROM mesures_opcua
WHERE time >= NOW() - INTERVAL '1 minute';
```

12 Orchestration avec Docker Compose

12.1 Objectif

Les six conteneurs ont d'abord été créés avec des commandes `docker run`. Une fois l'architecture stabilisée, ils ont été regroupés dans un fichier `compose.yaml` afin de rendre le laboratoire plus simple à démarrer, arrêter et reconstruire.

Les services gérés sont :

```
mosquitto
opcplc
node-red
influxdb3
timescaledb
grafana
```

Le réseau `industrial-net` et le volume `grafana-storage` existaient déjà. Ils sont donc déclarés comme ressources externes dans Compose.

```
networks:
  industrial-net:
    external: true
    name: industrial-net

volumes:
  grafana-storage:
    external: true
    name: grafana-storage
```

Le mot de passe TimescaleDB n'est pas écrit en clair dans `compose.yaml`. Il est chargé depuis un fichier `.env` local via `TIMESCALE_PASSWORD`. Un fichier `.env.example` sans secret est conservé avec la sauvegarde du projet.

12.2 Validation de la configuration

Avant le démarrage :

```
docker compose config --quiet && \
  echo "Configuration Compose valide"
```

Le démarrage complet est réalisé avec :

```
docker compose up -d
```

Puis l'état est contrôlé avec :

```
docker compose ps
```

Les six services ont été retrouvés en état `Up`, et Node-RED en état `healthy`.

La vue Docker Desktop regroupe les conteneurs du projet Compose `industrial_lab` et permet de visualiser rapidement leur état d'exécution et les ports publiés.

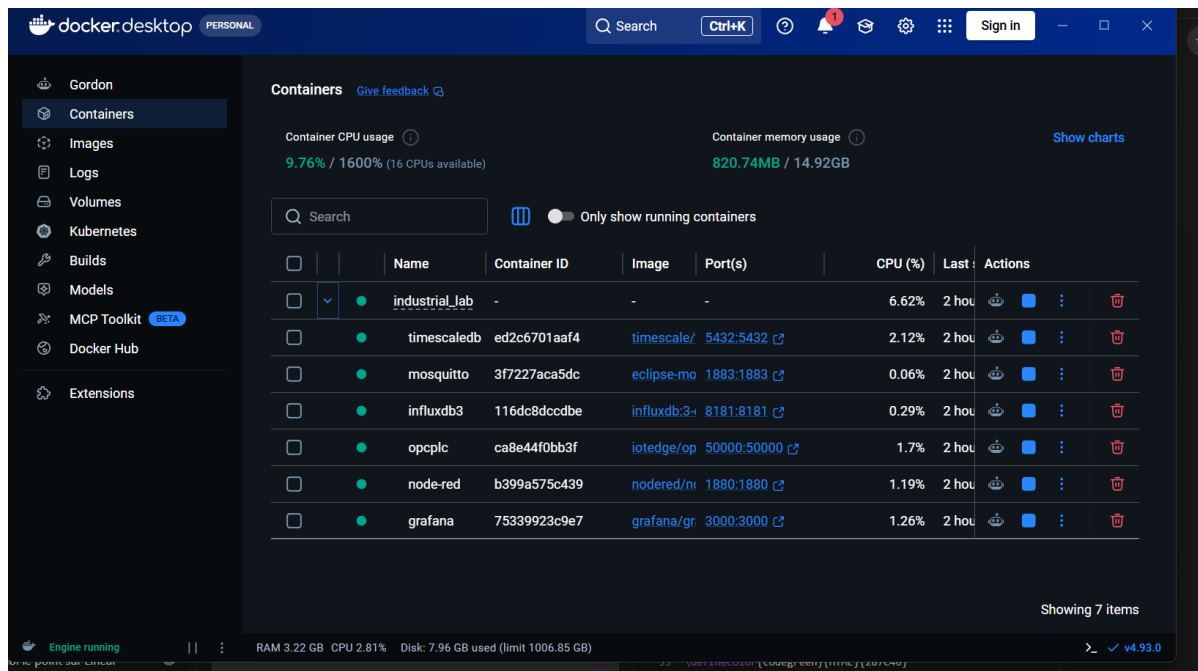


FIGURE 4 – Projet `industrial_lab` dans Docker Desktop avec les six services du laboratoire en exécution.

Le même état peut être contrôlé depuis le terminal avec `docker compose ps`. Cette vérification confirme notamment le statut `healthy` de Node-RED et les liaisons de ports limitées à l'interface locale.

```
sekou@DaikiPc:~/industrial_lab$ docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
grafana	grafana/grafana:latest	"/run.sh"	grafana	2 hours ago	Up 2 hours	127.0.0.1:3000->3000/tcp
influxdb3	influxdb:3-core	"/usr/bin/entrypoint..."	influxdb3	2 hours ago	Up 2 hours	127.0.0.1:8181->8181/tcp
mosquitto	eclipse-mosquitto:latest	"/docker-entrypoint..."	mosquitto	2 hours ago	Up 2 hours	127.0.0.1:1883->1883/tcp
node-red	node-red/node-red:latest	"/.entrypoint.sh"	node-red	2 hours ago	Up 2 hours (healthy)	127.0.0.1:1880->1880/tcp
opcplc	mcr.microsoft.com/iotedge/opc-plc:latest	"/.opcplc --pn=50000..."	opcplc	2 hours ago	Up 2 hours	127.0.0.1:50000->50000/tcp
timescaledb	timescale/timescaledb:2.30.1-pg17	"docker-entrypoint.s..."	timescaledb	2 hours ago	Up 2 hours	127.0.0.1:5432->5432/tcp

FIGURE 5 – Vérification des six services avec `docker compose ps`.

12.3 Utilisation quotidienne

Pour un usage quotidien, les conteneurs existants sont simplement arrêtés et redémarrés :

```
docker compose stop
docker compose start
```

Deux scripts ont également été créés :

```
~/industrial_lab/start_lab.sh
~/industrial_lab/stop_lab.sh
```

Ils se placent automatiquement dans le dossier du projet, exécutent la commande Compose puis affichent l'état des services.

13 Sauvegarde du laboratoire

13.1 Dossier de sauvegarde

Un dossier dédié a été créé :

```
~/industrial_lab_backup
```

Il contient notamment :

- `flows.json` : export du flow Node-RED ;
- le dashboard Grafana exporté en JSON ;
- `mosquitto.conf` ;
- `timescaledb_schema.sql` ;
- les fichiers `docker inspect` des services ;
- `industrial-net_inspect.json` ;
- `compose.yaml` ;
- `.env.example` ;
- `start_lab.sh` et `stop_lab.sh` ;
- un `README.txt` décrivant le contenu.

Le vrai fichier `.env` n'est pas inclus dans la sauvegarde destinée à être partagée, car il contient un secret.

13.2 Archive et copie Windows

Le dossier est compressé avec :

```
tar -czf ~/industrial_lab_backup_2026-10-02.tar.gz \\  
-C ~ industrial_lab_backup
```

Une copie est ensuite placée dans les téléchargements Windows via le montage WSL `/mnt/c/...`

L'intégrité de la copie a été vérifiée avec `sha256sum` : les deux archives présentaient le même condensat SHA-256.

14 Problèmes rencontrés et solutions

14.1 Connexion MQTT impossible

Le nœud MQTT affichait initialement un voyant jaune avec le statut **Connexion**.

L'analyse des journaux Node-RED a permis d'identifier l'erreur suivante :

```
Connection failed to broker:  
mqtt://mosquitto:1888
```

Le port configuré était incorrect.

Le broker Mosquitto écoutait sur le port 1883, et non sur le port 1888.

Après correction dans la configuration du broker Node-RED, le statut est passé à **connecté**.

14.2 Connexion OPC UA

La connexion initiale au simulateur a nécessité une mise en cohérence de l'endpoint annoncé par le serveur et de celui utilisé par Node-RED.

Le simulateur a été configuré avec un nom d'hôte explicite :

```
--hostname opcplc  
--plchostname=opcplc  
--certdnsnames=opcplc
```

La connexion fonctionnelle utilise :

```
opc.tcp://opcplc:50000
```

avec le mode de sécurité **SignAndEncrypt** et la politique **Basic256Sha256**.

14.3 Erreur lors du parcours OPC UA

La première exploration utilisait `ns=0;i=86`.

Cet identifiant correspond au dossier Types, ce qui expliquait l’affichage d’éléments comme `ReferenceTypes`, `DataTypes` ou `VariableTypes`.

Le parcours des données applicatives devait commencer par :

```
ns=0;i=85
```

qui correspond au dossier Objects.

14.4 Absence de valeurs dans MQTT

Après la connexion au broker, la publication ne transmettait pas encore les valeurs attendues.

Le contrôle des sorties du nœud `OpcUa-Client` a permis de distinguer la sortie contenant la mesure de celle qui renvoyait `undefined`.

La publication MQTT a été reliée à la sortie fournissant effectivement les données `StepUp`.

La réception a ensuite été confirmée avec `mosquitto_sub`.

14.5 Décalage temporel InfluxDB 3

Les premières écritures InfluxDB 3 contenaient des valeurs récentes mais des timestamps décalés de plusieurs heures. Les horloges UTC des conteneurs Node-RED et InfluxDB étaient pourtant correctes.

Le problème a été contourné de manière déterministe en forçant le timestamp transmis au nœud InfluxDB à partir de `msg.sourceTimestamp` fourni par OPC UA :

```
msg.timestamp = new Date(msg.sourceTimestamp);
```

Après cette modification, les timestamps InfluxDB et TimescaleDB sont devenus cohérents et les deux courbes ont pu être affichées simultanément dans Grafana.

14.6 Mélange des variables dans un topic MQTT

Après l’ajout de `RandomSignedInt32`, le nœud MQTT utilisait encore un topic fixe. Les valeurs des deux variables étaient donc publiées dans le même topic.

La création d’un routage dynamique basé sur `msg.topic` a permis d’obtenir :

```
industrie/opcua/StepUp  
industrie/opcua/RandomSignedInt32
```

15 Sécurisation du laboratoire

15.1 Limitation de l’exposition réseau

Lors de l’installation initiale, les ports étaient publiés sur toutes les interfaces réseau.

Par exemple :

```
0.0.0.0:1883->1883/tcp
```

Cette exposition n’était pas nécessaire, car les services communiquent entre eux via `industrial-net`. Les conteneurs ont donc été recréés avec une publication limitée à l’interface locale IPv4.

Service	Publication finale
Node-RED	127.0.0.1:1880:1880
Mosquitto	127.0.0.1:1883:1883
OPC PLC	127.0.0.1:50000:50000
InfluxDB 3	127.0.0.1:8181:8181
TimescaleDB	127.0.0.1:5432:5432
Grafana	127.0.0.1:3000:3000

TABLE 5 – Ports publiés sur l'interface locale

Cette modification limite l'exposition des ports publiés par Docker sans interrompre les échanges internes entre les trois conteneurs.

15.2 Contrôle des ports

La vérification a été réalisée avec :

```
docker ps --format \
"table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

Les six services du laboratoire sont publiés uniquement sur l'interface locale 127.0.0.1. Node-RED affiche également l'état **healthy** après son démarrage.

Les conteneurs de démonstration et d'essai inutiles ont été supprimés après vérification de l'absence de volumes persistants associés.

15.3 Limites de sécurité

Cette installation constitue un environnement d'apprentissage et non une architecture prête pour une mise en production.

En particulier :

- MQTT utilise temporairement une connexion anonyme ;
- le simulateur OPC UA accepte automatiquement les certificats clients ;
- l'interface d'administration Node-RED doit être protégée avant toute exposition externe ;
- les versions des images Docker devront être explicitement figées pour assurer la reproductibilité à long terme.

Avant un déploiement réel, il faudra notamment mettre en place une authentification adaptée, une gestion des certificats et une politique d'accès réseau.

16 Commandes utiles de maintenance

16.1 Visualiser les conteneurs

```
docker ps
```

16.2 Consulter les journaux Node-RED

```
docker logs --tail 40 node-red
```

16.3 Consulter les journaux OPC PLC

```
docker logs --tail 40 opcplc
```

16.4 Contrôler le réseau industriel

```
docker network inspect industrial-net
```

16.5 Arrêter les services

```
cd ~/industrial_lab
docker compose stop
```

Ou directement :

```
~/industrial_lab/stop_lab.sh
```

16.6 Redémarrer les services existants

```
cd ~/industrial_lab
docker compose start
```

Ou directement :

```
~/industrial_lab/start_lab.sh
```

Ces commandes arrêtent ou redémarrent les conteneurs sans supprimer les données persistantes.

17 Bilan des validations

Test réalisé	Résultat
Démarrage des six services avec Docker Compose	OK
Communication MQTT locale	OK
Connexion au serveur OPC UA	OK
Navigation dans l'arbre OPC UA	OK
Lecture de StepUp et RandomSignedInt32	OK
Lecture cyclique proche de 0,5 Hz	OK
Routage MQTT dynamique par variable	OK
Écriture dans InfluxDB 3	OK
Écriture dans TimescaleDB	OK
Cohérence des timestamps entre les deux bases	OK
Dashboard Grafana historique	OK
Indicateurs d'état et de fraîcheur Grafana	OK
Limitation des ports à localhost	OK
Scripts start/stop et sauvegarde du labo	OK

TABLE 6 – Synthèse des essais réalisés

18 Perspectives

La chaîne d'acquisition, d'historisation et de supervision est maintenant fonctionnelle. Les travaux suivants pourront porter sur :

- la création d'alarmes Grafana et de règles de notification ;
- la journalisation des événements et défauts ;
- l'ajout d'autres variables OPC UA et de tags machine ;
- l'authentification MQTT et le durcissement des accès ;

- le gel explicite des versions des images Docker ;
- l'exploitation des historiques avec Python ;
- la détection d'anomalies ou la maintenance prédictive par intelligence artificielle.

19 Conclusion

Ce laboratoire a permis de construire une chaîne complète d'acquisition, de transmission, d'historisation et de supervision de données industrielles.

Deux variables OPC UA sont lues par Node-RED, publiées sur des topics MQTT séparés, enregistrées dans InfluxDB 3 et TimescaleDB puis visualisées dans Grafana.

L'orchestration par Docker Compose, la persistance des données, les scripts de démarrage et d'arrêt ainsi que la stratégie de sauvegarde rendent l'environnement réutilisable et reproductible pour la poursuite des travaux d'automatisme, de supervision et d'analyse de données.